

OBJECT-ORIENTED PROGRAMMING (OOP) WEEK-1

Instructor:

Nida Munawar

Lecturer

(Computer Science
Department)

National University- FAST
(KHI Campus)

GCR CODE



ACKNOWLEDGMENT

Publish

- Publish material by Virtual University of Pakistan.

Publish

- Publish material by Deitel & Deitel.

Publish

- Publish material by Robert Lafore.

WHAT YOU HAVE DONE SO FAR!

- Programming Fundamentals (PF):
 - How to think a program.
 - How to write a program.
 - Basic Programming structure.
 - Procedural paradigm.
 - Group of functions that interact with each other.
- Already have knowledge about the offered course (Repeater's Course).
- Need to strengthen our concepts.
- Try to implement what we know.

```

mirror_mod = modifier_ob.mirror_mod
# Set mirror object to mirror
mirror_mod.mirror_object = mirror_ob

# operation == "MIRROR_X":
mirror_mod.use_x = True
mirror_mod.use_y = False
mirror_mod.use_z = False
# operation == "MIRROR_Y":
mirror_mod.use_x = False
mirror_mod.use_y = True
mirror_mod.use_z = False
# operation == "MIRROR_Z":
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True

# selection at the end -add
mirror_ob.select= 1
modifier_ob.select=1
context.scene.objects.active = mirror_ob
print("Selected" + str(modifier_ob.name))
mirror_ob.select = 0
# bpy.context.selected_objects[0] = mirror_ob
# bpy.data.objects[one.name].select_set(True)

print("please select exactly one mirror")

-- OPERATOR CLASSES -----

bpy.types.Operator):
    """Set mirror to the selected object.mirror_mirror_x"
    mirror X"

    context):
        context.active_object is not None

```

OO PROGRAMMING AS A COURSE

What we will study:

Object Oriented Programming.

How to think in a OOP way.

How to map real world into a program

Or, how to program a real world scenario.

Aim :

Our aim is to learn the concepts of Object Oriented programming.

Try to digest them.

Implement in a program.

Tool: C++.



CONTENTS OF THE COURSE

- Object Oriented Programming.
- Classes & Objects.
- Overloading.
- Inheritance.
- Polymorphism.
- Generic Programming.
- Exception Handling.

BOOKS



Text Book:



1- C++ How to program By Deitel & Deitel.



Reference Books:



1- The C++ Programming Language By Bjarne Stroustrup.



2- Object Oriented Software Engineering By Jacobson.

GRADING POLICY

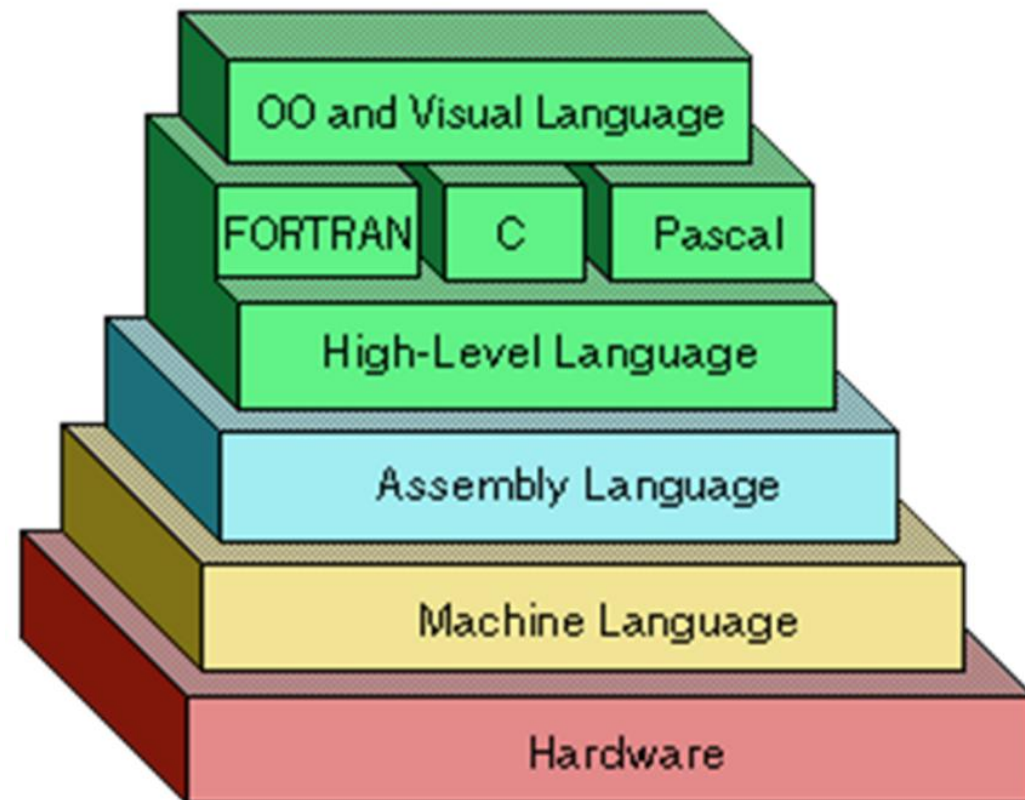
Assignments	10	at least three
Quiz	10	at least three(or N-1)
Midterm's	30	two (15 marks each)
Final	50	
Total	100	

WEEK ONE, CLASS ONE

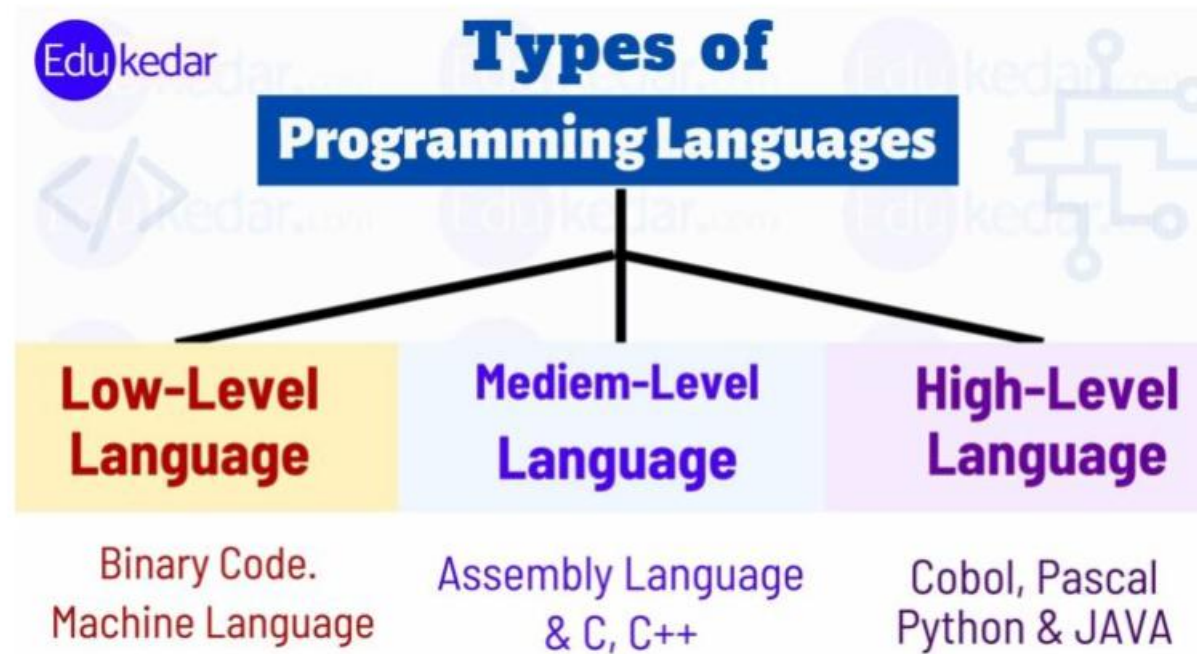
- Introduction to the Generation of Languages

- Introduction to Programming Paradigms

WHAT ARE THE TYPES OF PROGRAMMING LANGUAGES



TYPES OF PROGRAMMING LANGUAGES



WHAT ARE THE TYPES OF PROGRAMMING LANGUAGES

First Generation Languages

Second Generation Languages

Third Generation Languages

Fourth Generation Languages

Fifth Generation Languages

FIRST GENERATION LANGUAGES

Machine language

- **Operation code** – such as addition or subtraction.
- **Operands** – that identify the data to be processed.
- Machine language is machine dependent as it is the only language the computer can understand.
- Very efficient code but very difficult to write.
- (e.g., 01011100)

+1300042774

+1400593419

+1200274027 adds overtime pay to base pay and
stores the result in gross pay:

SECOND GENERATION LANGUAGES

Assembly languages

- Symbolic operation codes replaced binary operation codes.
- Assembly language programs needed to be “assembled” for execution by the computer. Each assembly language instruction is translated into one machine language instruction.
- Very efficient code and easier to write.
- (e.g., **ADD**, **MOV**)

load basepay

add overpay

store grosspay

THIRD GENERATION LANGUAGES

High Level Languages

Closer to English but included simple mathematical notation.

- Programs written in **source code** which must be translated into machine language programs called **object code**.
- The translation of source code to object code is accomplished by a machine language system program called a **compiler**.
- `grossPay = basePay + overTimePay;`

THIRD GENERATION LANGUAGES (CONT'D.)

Alternative to compilation is interpretation which is accomplished by a system program called an interpreter.



Common third generation languages

FORTRAN

COBOL

C and C++

Visual Basic

FOURTH GENERATION LANGUAGES

Event based programming

A high level language (4GL) that requires fewer instructions to accomplish a task than a third generation language.

Used with databases

- Query languages
- Report generators
- Forms designers
- Application generators



FIFTH GENERATION LANGUAGES

Artificial Intelligence

Machines that can think like humans

Machines that takes its own decisions

PROGRAMMING PARADIGMS



The term **paradigm** = method to solve a problem



The term **programming paradigm** is used to specify an overall approach to writing program code

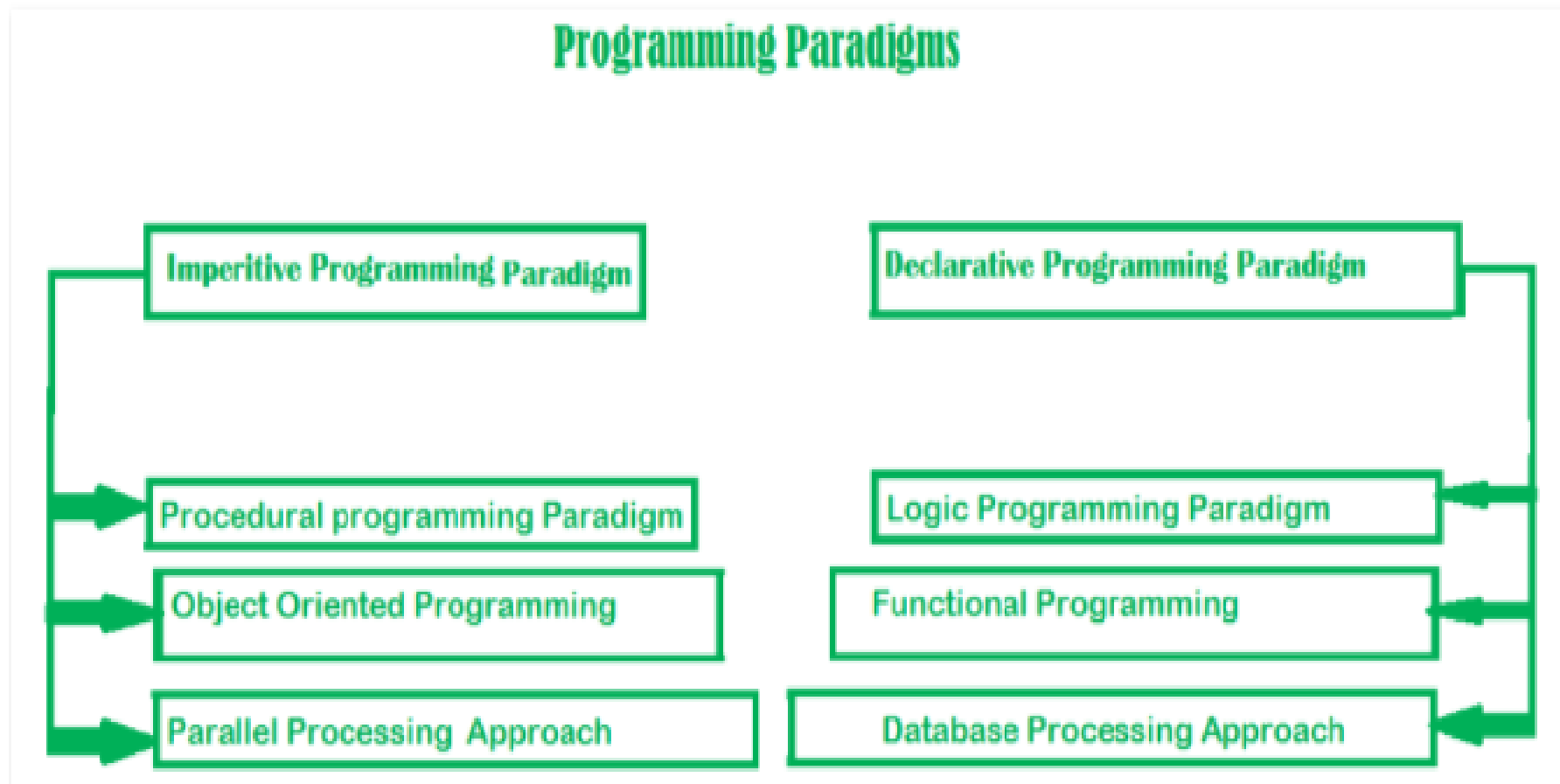


Many programming languages support multiple paradigms. Python, which is a popular general-purpose programming language, allows you to code using procedural, object-oriented.



In the early days of computer programming, most programs comprised a single procedure with many lines of code.

PROGRAMMING PARADIGMS



THE C++ LANGUAGE FEATURES MOST DIRECTLY SUPPORT FOUR PROGRAMMING STYLES

Paradigm.

Procedural programming

PP can be defined as a programming model which is derived from structured programming, based upon the concept of calling procedure. Procedures, also known as routines, subroutines or functions

Data abstraction

This is programming focused on the design of interfaces, hiding implementation details in general and representations

OOP programming

OOP can be defined as a programming model which is based upon the concept of objects

Generic programming

This is programming focused on the design, implementation, and use of general algorithms.

WHAT IS POP

A procedural language is a sort of computer programming language that has a set of functions, instructions, and statements that must be executed in a certain order to accomplish a job or program.

Procedural languages include BASIC, C, FORTRAN, and Pascal, to name a few.

features:

- a. Modularity
- b. Predefined Function
- c. Scoping

Advantages

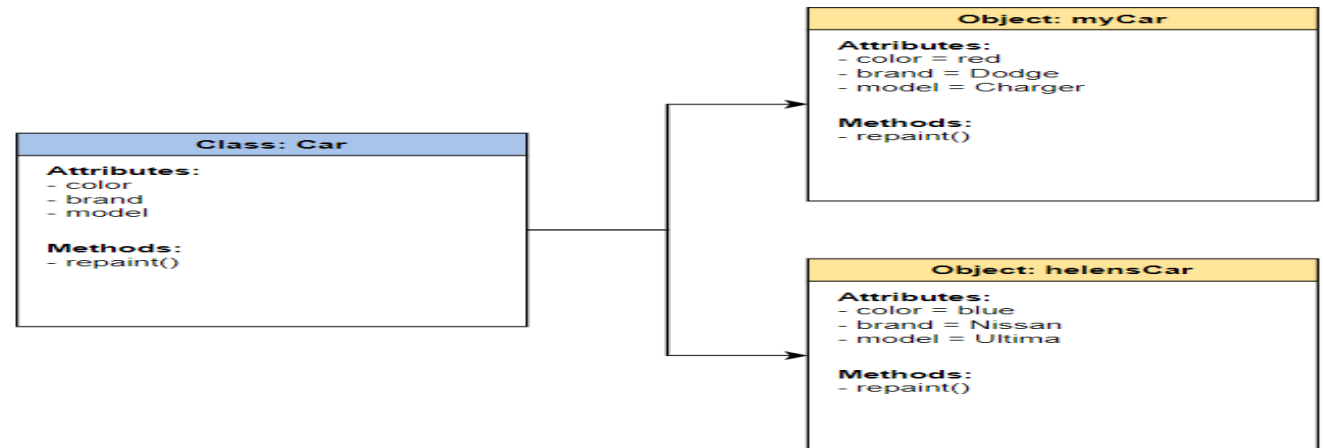
- 1. portable
- 2. Function
- 3. Flow Tracking

Disadvantages

- large scale
- not work for real world problem
- Data Security

LET'S TALK ABOUT OOP

- Object-Oriented Programming (OOP) is a programming paradigm in computer science that relies on the concept of classes and objects.
- It is used to structure a software program into simple, reusable pieces of code blueprints (usually called classes),
- which are used to create individual instances of objects.



PROGRAMMING PARADIGMS

The **procedural programming paradigm** is where program code is divided up into **procedures**, discrete blocks of code that carry out a single task. **procedural programming paradigm** sometimes referred to as **top-down** problem-solving

Splitting code into smaller chunks has many benefits:

- It is much easier to test and debug, e.g. tracing 20 lines of code instead of 200 lines of code.
- **procedures** can be called many times, reducing the amount of repeated code.
- **procedures** can manipulate shared data.

PROCEDURAL VS OBJECT ORIENTED

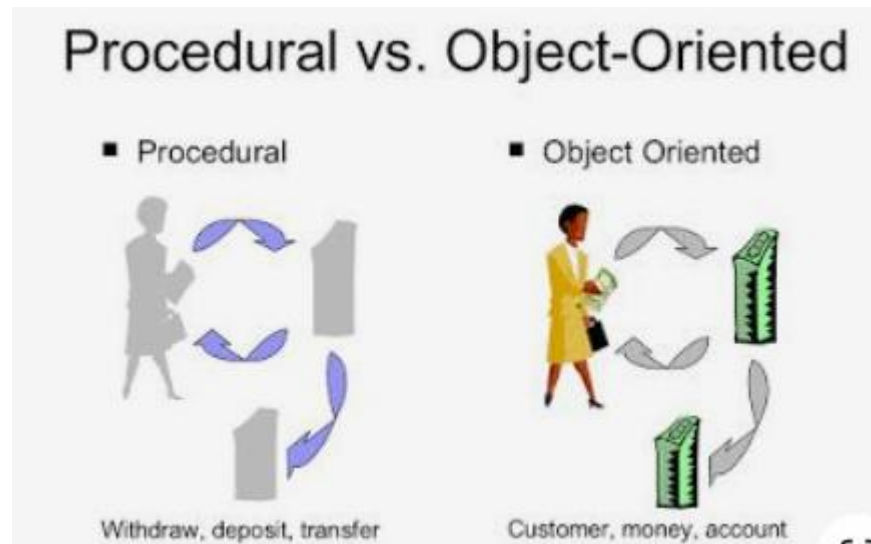
Data integrity

One criticism of the procedural paradigm is that it focuses on what needs to be done, rather than on the integrity of the data that it manipulates. The problem with global variables is that, since they can be accessed and modified by every subroutine in a program

procedure oriented programming languages **follows top down approach**

object oriented programming languages **follows bottom up approach**

PROCEDURAL VS OBJECT ORIENTED



PROGRAMMING PARADIGMS

The **object-oriented programming paradigm (OOP)** introduces a fundamentally different approach to problem-solving. Rather than focusing on the problem that needs to be solved, OOP focuses on the **objects** that make up the system.

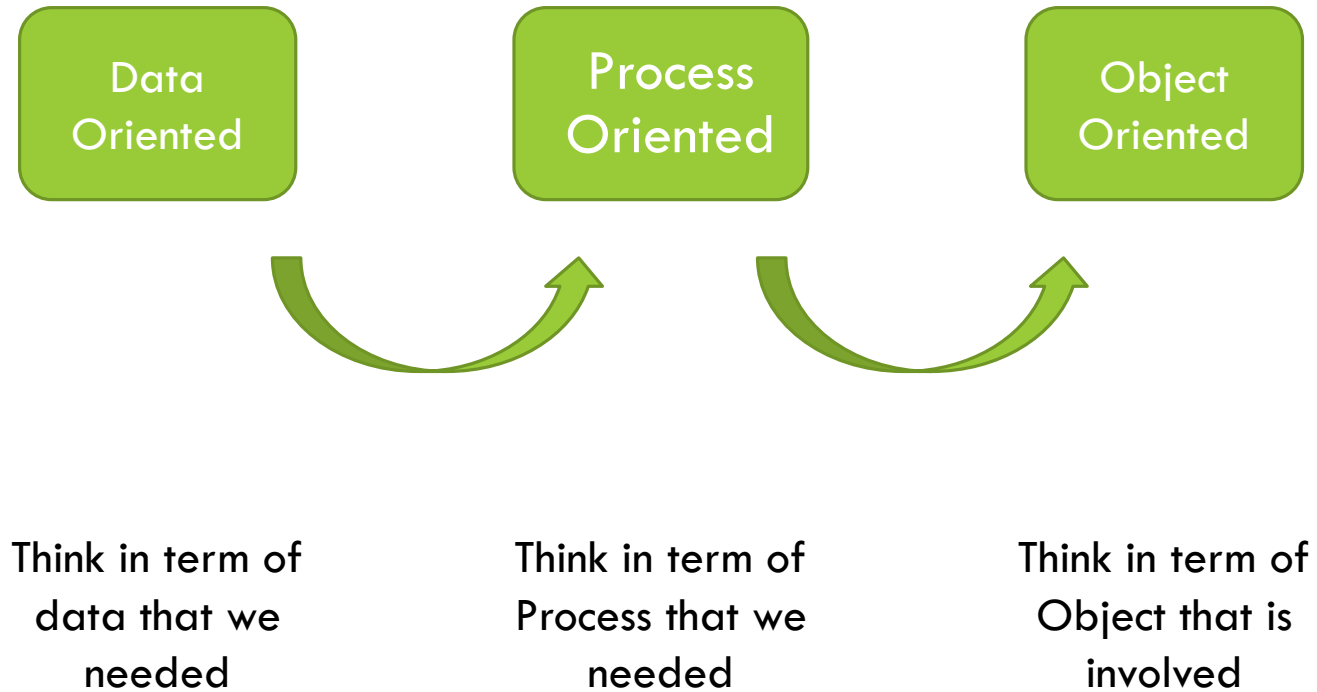
1- Sequential

2- Procedural

3- Object Oriented

PROGRAMMING PARADIGMS

PROGRAMMING PARADIGMS



WEEK ONE, CLASS TWO

- Introduction to OO paradigm
- Principles of Object Oriented Paradigm

WHAT IS OBJECT ORIENTATION

- A technique for system modeling.
- OO model consists of several interacting objects.

WHAT IS A MODEL?

- abstraction of something.
- Purpose is to understand the product before developing it.

EXAMPLE — OO MODEL



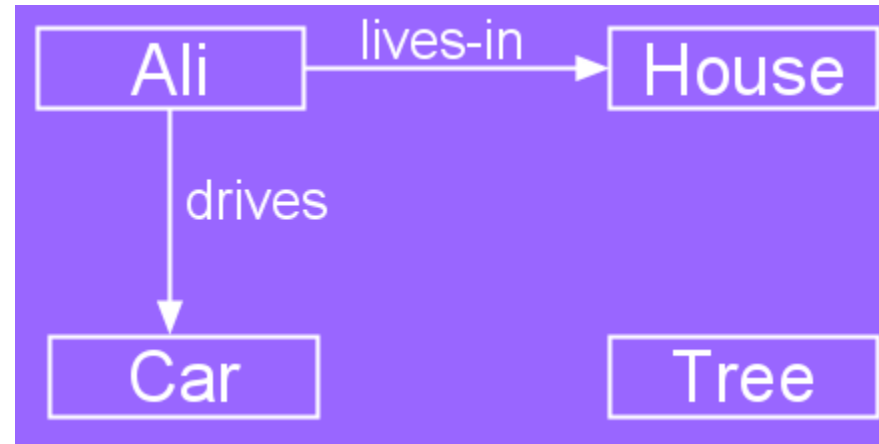
EXAMPLE — OO MODEL

Objects

- Person i.e Name: Ali
- House
- Car
- Tree

Interactions

- Ali lives in the house
- Ali drives the car



OBJECT-ORIENTATION - ADVANTAGES

- People think in terms of objects
- OO models map to reality

Therefore, OO models are:

- easy to develop
- easy to understand

FIVE PRINCIPLES OF OO PARADIGM

- 1- Abstraction: To have the relevant information.
- 2- Encapsulation: To hide information inside the object.
- 3- Polymorphism: To have many shapes / behaviors.
- 4- Inheritance: To create a new object with an existing one (To adopt features from others)
- 5- Reusability: Ability to use an object again and again if needed.

FIVE PRINCIPLES OF OO PARADIGM

- 1- Abstraction: To have the relevant information.
- 2- Encapsulation: To hide information inside the object.
- 3- Polymorphism: To have many shapes / behaviors.
- 4- Inheritance: To create a new object with an existing one (To adopt features from others)
- 5- Reusability: Ability to use an object again and again if needed.

INHERITANCE

Inheritance is a features of OOP which allow the user to inherit the common properties from other class

For example, Class Vehicle and class bike and Class car

Types of inheritance:

1. Single
2. Multiple
3. Multivel
4. Hierarchical
5. Hybrid

POLYMORPHISM

polymorphism refers to ability to exist in multiple form. Multiple interface can be given to a single interface

For example: First, define an abstract class named Person that has an abstract method called greet().

Type f Polymorphism:

- a. Static / Compile time (Overloading)
- b. Dynamic/ Run time (Overriding)

ENCAPSULATION

Encapsulation refers to binding the data and code that works on together in single unit.

For example: we have a class named as car and we have all the member functions and member variables wrapped inside this single unit and this binding of data is known as encapsulation.

Access Modifier:

- ← Public
- ← Private
- ← Protected

ABSTRACTION

Abstraction concept only allowing to display the important information and hide the implementation details from user.

Achieve by:

- Abstract Class
- Abstract Method

REAL-LIFE EXAMPLE

- We can take a real-life example of an Air conditioner (AC).
- We have a remote control in order to control the various AC functions like start, stop, increase/decrease the temperature, control humidity, etc.
- We can control these functions just with a click of a button but internally there is a complex logic that is implemented to perform these functions.

DATA ABSTRACTION (EXAMPLE)



Real Life Example of Abstraction

OOP PRINCIPLE

OOP Principles

Encapsulation

When an object only exposes the selected information.

Abstraction

Hides complex details to reduce complexity.

Inheritance

Entities can inherit attributes from other entities.

Polymorphism

Entities can have more than one form.



WEEK ONE, CLASS THREE

- Introduction to Object

WHAT IS AN OBJECT?

An object is:

- It can be anything for which we want to save Information
- Something tangible (Ali, Car)
- Something that can be captured intellectually (Time, date)

An object has:

- State / attributes / properties / data
- Well-defined behavior / methods / functions
- Unique identity

ALI AS AN OBJECT

Attributes:

- Name
- age

Behavior (operations)

- Walks
- Eats

Identity

- His name

CAR AS AN OBJECT

State (attributes)

- Color
- Model

Behavior (operations)

- Accelerate
- Start Car
- Change Gear

Identity

- Its registration number

BOARD MARKER AS AN OBJECT

Attributes?

Behavior?

VISUALIZING AN OBJECT

